TEKNOFEST 2026

MESLEKİ YETENEK YARIŞMASI

AKILLI FABRİKA SİSTEMLERİ PROGRAMLAMA KATEGORİSİ

TEKNİK ŞARTNAME

2026

İÇİNDEKİLER

TANIMLAR VE KISALTMALAR 3
1. AMAÇ VE KAPSAM 4
2. DONANIM ALTYAPISI 4
3. SİSTEM GENEL TANIMI 5
3.1 Kumanda panosu ve Konveyör 5
3.2. Robot Kol 5
3.3. Otonom Araç 5
4. SAĞLANAN İÇERİKLER 5
5. YARI FİNAL GÖREVLERİ VE TESLİMAT 6
5.1. Görev 1 – Sistem Algoritması Tasarımı (Robot Kol + Otonom Araç) 6
5.2. Görev 2 – Arayüz Tasarımı ve Step Motor Kontrolü 7
5.3. Görev 3 – HSV ile Renk Algılama 8
5.4. Görev 4 – MQTT ile Haberleşme 8
5.5. Görev 5 – YOLO ile Tabela Tespiti Modeli 8
5.6. Görev 6 – PLC ve HMI Programı 9
5.7. Görevlerde Kullanılacak Kütüphaneler ve Sürümler 9
5.8. Teslim Formatı 10
6. PUANLAMA 10
7. FİNAL AŞAMASI HAKKINDA 11
8. KURALLAR 11
9. İLETİŞİM 11

TANIMLAR VE KISALTMALAR

Akıllı Fabrika: PLC, robotik, görüntü işleme ve otonom taşıma sistemlerinin entegre çalıştığı endüstriyel otomasyon kurgusu.

Görüntü İşleme / Yapay Zekâ: Kamera verisinden anlam çıkarma, algılama ve karar verme amaçlı algoritmalar.

HMI (Human Machine Interface) / Operatör Paneli: Operatörün üretim hattını başlatmak, izlemek, durdurmak ve alarmları yönetmek için kullandığı; PLC ile haberleşen endüstriyel kullanıcı arayüzü. Bu yarışmada KTP700 Basic kullanılmaktadır.

Jetson Orin Nano: NVIDIA tarafından üretilen, robot kol ve otonom araç yazılımlarının çalıştırıldığı yapay zekâ geliştirme kartı.

KYS: TEKNOFEST Kurumsal Yönetim Sistemi (başvuru, dosya yükleme ve duyuru platformu).

Ladder (Merdiven Diyagramı): PLC programlama dili.

PLC: Programlanabilir Lojik Kontrolör. Bu yarışmada Siemens S7-1200 1214C DC/DC/DC kullanılmaktadır.

TIA Portal: Siemens otomasyon ürünlerinin programlandığı geliştirme ortamı; bu yarışmada V14 ve üzeri sürümler kabul edilir.

V20: Siemens SINAMICS V20 frekans inverteri (sürücü); bu yarışmada konveyör motorunun sürülmesi için kullanılır.

YOLO: You Only Look Once; gerçek zamanlı nesne tespiti için kullanılan derin öğrenme tabanlı algoritma.

MZ80: Kızılötesi cisim algılama sensörü; otonom araç üzerinde bulunur ve küpün araca yüklendiğini algılar.

MQTT: Haberleşme protokolü; Wi-Fi üzerinden TCP/IP ile haberleşme sağlar. Bu yarışmada robot kolun küp rengini otonom araca bildirmesi için kullanılır.

1. AMAÇ VE KAPSAM

Bu teknik şartname; TEKNOFEST 2026 Mesleki Yetenek Yarışması Akıllı Fabrika Sistemleri Programlama Kategorisi kapsamında; sistemin genel tanımını, yarı final görevlerini, takımlara sağlanacak içerikleri, teslim formatını, puanlama esaslarını ve final aşamasına ilişkin genel bilgileri içermektedir.

Bu doküman, yarışmanın Genel Şartnamesi ile birlikte uygulanır. Genel Şartname ile bu Teknik Şartname arasında çelişki olması durumunda, teknik konularda bu Teknik Şartname; süreç ve katılım kuralları konusunda Genel Şartname esas alınır.

Yarışmacılardan; yarı final aşamasında bu şartnamede tanımlı görevleri eksiksiz olarak teslim etmeleri; final aşamasında ise üretim hattının PLC kontrolü, HMI operatör arayüzü, robot kolun renk algılamalı pick-and-place görevi ve otonom aracın şerit takipli görev senaryosundan oluşan, organizasyon tarafından çalışır durumda sağlanan sistem üzerinde istenen revizyon ve ek görevleri uygulamaları beklenmektedir.

2. DONANIM ALTYAPISI

BİLEŞEN	MODEL / TİP	İŞLEV
PLC	Siemens S7-1200 CPU 1214C DC/DC/DC (6ES7214-1AG40-0XB0)	Hat kontrolü
HMI	Siemens KTP700 Basic (6AV2123-2GB03-0AX0)	Operatör arayüzü
Motor Sürücü	Siemens SINAMICS V20 (0,75 kW) (6SL3210-5BB17-5UV1)	Konveyör motor sürücüsü
Konveyör Motoru	3 fazlı asenkron motor, 0,18 kW	Bant tahriki
Sensörler	Optik sensör OMRON E3FA-RP11	Küp algılama
Ana İşlemci (Robot/Araç)	NVIDIA Jetson Orin Nano Developer Kit	AI + görüntü işleme
Robot Kol Kamerası	Raspberry Pi Camera v2 (CSI/Flex bağlantı)	Renk algılama
Robot Kol Motor Kontrolü	Arduino Uno + DM556 benzeri step motor sürücüleri	4 eksen kontrolü / PLC den gelecek başla sinyali
Otonom Araç malzeme algılama	MZ80	Küp algılama
Otonom Araç Kamerası	Intel RealSense D455	Şerit & tabela tanıma
Otonom Araç İşlemcisi	ESP32 (USB ile Jetson’a bağlı)	Motor/servo & buton

3. SİSTEM GENEL TANIMI

Sistem; bir konveyör, üzerinde kamera bulunan bir robot kol, Jetson tabanlı bir otonom araç, hattı yöneten bir PLC ve operatöre arayüz sunan bir kumanda paneli (HMI) bileşenlerinden oluşan minyatür bir akıllı fabrika hattıdır.

Özetle çalışma şöyledir: Operatör, kumanda paneli üzerinden hattı başlatır. Konveyör çalışır ve gelen küp konveyör çıkışında algılandığında konveyör durur. Robot kol küpü alır ve otonom araca yükler. Araç, pistteki görevini yapmak üzere harekete başlar ve görevini tamamlayarak park eder. Sistem bileşenlerinin çalışma detayları aşağıda verilmiştir.

3.1 Kumanda panosu ve Konveyör

Start butonuna basıldıktan sonra konveyör girişinde küp var ise sistem çalışmaya başlayacak ve yeşil sinyal lambası aktif olacaktır. Konveyör çıkışında sensörün algılama yapmasıyla beraber konveyör duraklayacak otonom araç yükleme noktasında ise robot kola başla sinyali gönderilecektir. Konveyör durakladığında yeşil sinyal lambası sönecek, kırmızı sinyal lambası aktif olacaktır. Robot kol küpü alıp araca yükleyecek (renk tespiti robot koldaki kamera ile yapılacaktır) ve otonom araç küp üzerinde iken saha görevlerini gerçekleştirip küp rengine uygun park alanına girecektir.(Pano bağlantı şeması incelenmelidir)

3.2. Robot Kol

Konveyör çıkışında küp algılandığında ve otonom araç yükleme noktasında hazır olduğunda, PLC robot kola BAŞLA sinyali gönderir. Robot kol küpün rengini (kırmızı / yeşil / mavi) KLASİK GÖRÜNTÜ İŞLEME (HSV) ile belirler; robot kolda yapay zekâ / model KULLANILMAZ. Kol küpü otonom araca yükler ve küpün rengini MQTT ile araca bildirir (arac/yuk konusu; RED / GREEN / BLUE). PLC, konveyör ve robot kol; konveyöre küp geldikçe bu çevrimi sürekli tekrarlayacak şekilde tasarlanmıştır.

3.3. Otonom Araç

Aracın hareket edebilmesi için İKİ ŞARTIN BİRLİKTE sağlanması gerekir: (1) araç üzerindeki MZ80 sensörünün küpü algılaması ve (2) MQTT üzerinden renk bildiriminin gelmesi. Şartlardan yalnızca birinin sağlanması yeterli değildir. Hareketine başlayan araç, şerit üzerinde ilerlerken pistte yer alan GÖREVLERİ yerine getirir; görevler pist üzerindeki TABELALAR ile işaretlenir. Araç, küpün rengine uygun renkli park alanına park ederek görevini tamamlar; otonom araç tek görevliktir, park ile birlikte durur.

Tabela tanıma, yarışmacının kendi eğittiği YOLO modeliyle yapılır (bk. Görev 5).

4. SAĞLANAN İÇERİKLER

Yarı finale geçen takımlara KYS üzerinden YALNIZCA tabela görselleri (tabelalar.zip) iletilecektir; bu aşamada takımlara başka kod, model veya iskelet dosya gönderilmez. Tabela görselleri, Görev 5 kapsamındaki YOLO modelinin eğitiminde kullanılacaktır. Finale kalan

takımlara sağlanacak yazılım paketleri ve detaylı yarışma senaryosu, final öncesinde ayrıca duyurulacaktır. Pano bağlantı şeması ek olarak verilmiştir.

5. YARI FİNAL GÖREVLERİ VE TESLİMAT

Yarı final değerlendirmesi, tamamen aşağıda tanımlı 5 (beş) görev üzerinden yapılır. Takımlara bu aşamada kod veya iskelet dosya gönderilmez; yalnızca tabela görselleri iletilir (bk. Bölüm 3). Tüm görev çıktıları, Bölüm 5.8'de tanımlı yapıya uygun TEK bir sıkıştırılmış dosya (.zip) olarak KYS üzerinden teslim edilir.

5.1. Görev 1 – Sistem Algoritması Tasarımı (Robot Kol + Otonom Araç)

Yarışmacıdan beklenen; robot kol ve otonom aracı BİRLİKTE kapsayan, sistemin tüm çevrimini gerçekleştiren TEK bir algoritmayı, yalnızca aşağıdaki hazır eylemleri kullanarak AKIŞ ŞEMASI ve SÖZDE KOD olarak tasarlamasıdır. Tasarım; robot kol sürecini, otonom araç sürecini ve ikisi arasındaki haberleşmeyi (renk bildirimi ve hareket şartları) birlikte göstermelidir. Kolun önceden öğretilmiş üç pozu vardır: GORME (konveyörü izleyip beklediği poz), AL (küpü kavradığı poz) ve YUKLE (küpü araca bıraktığı poz).

Robot kol için hazır eylemler:

HAZIR EYLEM	AÇIKLAMA
`home()`	**Tüm eksenleri referans konumuna götürür.**
`git("GORME"|"AL"|"YUKLE")`	**Önceden öğretilmiş poza gider.**
`tut()` / `birak()`	**Tutucuyu kapatır (küpü kavrar) / açar (küpü bırakır).**
`renk_oku()`	**"RED", "GREEN", "BLUE" veya "UNKNOWN" döndürür.**
`basla_geldi()`	**PLC başla sinyali (küp + araç hazır); True / False döndürür.**
`araca_gonder(renk)`	**Küpün rengini MQTT ile araca bildirir.**

Otonom araç için pistte yer alan görevler ve beklenen davranışlar şunlardır: trafik ışığı, yaya geçidi tabelası, tümsek tabelası, park tabelası.

Otonom araç için hazır eylemler:

HAZIR EYLEM	AÇIKLAMA
`yuk_rengi_bekle()`	**MQTT'den renk gelene kadar bekler; "RED", "GREEN" veya "BLUE" döndürür.**
`yuk_algilandi()`	**MZ80 sensörü küpü algıladıysa True döndürür.**
`ilerle()` / `dur()` / `bekle(sn)`	**Aracı ilerletir / durdurur / verilen süre kadar bekletir.**
`isik_oku()`	**"KIRMIZI", "YESIL" veya "YOK" döndürür.**

HAZIR EYLEM	AÇIKLAMA
`tabela_oku()`	**"YAYA", "TUMSEK", "PARK" veya "YOK" döndürür.**
`park_et(renk)`	**Verilen renkteki park alanına park eder.**

Tasarımda cevaplanması gereken kararlar:

- Başlangıç: Sistem ilk açıldığında, döngüye girmeden önce yapılması gereken bir şey var mı? Gerekçelendiriniz.
- Bekleme: Başla sinyalini beklemeyi nasıl kurgularsınız?
- Renk: Rengi ne zaman ve kaç kez okursunuz? UNKNOWN gelirse ne olur?
- Alma–Yükleme: Hangi sırayla yapılır?
- Bildirim: Rengi araca ne zaman gönderirsiniz?
- Sonraki küp: Kol geri döndüğünde sinyal hâlâ aktifse / değilse ne olur?
- Döngü: Süreç nasıl tekrarlanır; nerede başlar, nereye döner?
- Başlangıç: Araç ne zaman harekete başlar? İki şarttan (MZ80 + renk bildirimi) yalnızca biri sağlanırsa ne olur?
- Trafik ışığı: Kırmızıda beklemeyi nasıl kurgularsınız? Yeşile dönüşü nasıl anlarsınız?
- Tabelalar: Tabela görüldüğü an mı durulur, yaklaşınca mı? Aynı tabela iki kez görülürse ne olur?
- Park seçimi: Park yeri neye göre seçilir? Renk bilgisi hiç gelmezse araç ne yapar?
- Görev sonu: Park tamamlanınca ne olur?

Kurallar: Araç yerinde değilken yükleme yapılmamalıdır (başla sinyali bunu garantiler). MZ80 algılaması ve renk bildirimi BİRLİKTE sağlanmadan araç hareket etmemeli; kırmızı ışıkta kesinlikle geçilmemelidir.

Teslim: Akış şeması ve sözde kod, tek bir PDF dosyası (algoritma.pdf) olarak hazırlanır ve gorev1.zip içerisinde teslim edilir.

5.2. Görev 2 – Arayüz Tasarımı ve Step Motor Kontrolü

Tek bir step motoru kontrol eden masaüstü arayüz tasarlanıp kodlanacaktır. Arayüz Python (tkinter) ile yazılır; motor komutları pyserial üzerinden Arduino'ya gönderilir. Motor konumu adım (step) cinsinden takip edilir ve kaydedilen konumlar bir JSON dosyasına yazılır; program kapatılıp açıldığında kayıtlar kaybolmamalıdır.

Arayüzde bulunması zorunlu butonlar ve davranışları:

- BAŞLANGICI KAYDET: Motorun o anki konumunu "baslangic" olarak JSON'a yazar.
- BAŞLA: Motor rastgele yönde/hızda dönmeye başlar.
- DUR: Motor durur.
- KONUMU KAYDET: Motorun durduğu konumu "kayitli" olarak JSON'a yazar.
- BAŞLANGICA GİT: Motor, kayıtlı "baslangic" konumuna gider.
- KAYITLI KONUMA GİT: Motor, kayıtlı "kayitli" konumuna gider.

Arayüzde ayrıca motorun anlık konumu (adım) ve durumu (dönüyor/duruyor) görünmelidir.

Standart bağlantı (ZORUNLU): Kodlar organizasyonun test düzeneğinde çalıştırılacağından bağlantı standarttır: Motor sürücüsü DM556 (PUL/DIR girişli); PUL = Arduino D2, DIR = Arduino D3; mikro adım 1/8 (motor 1600 adım/tur). Bu bağlantıya uymayan kod test düzeneğinde çalıştırılamaz ve değerlendirilemez.

Kullanım senaryosu (bu sırayla çalışmalıdır): 1) BAŞLANGICI KAYDET, 2) BAŞLA, 3) DUR, 4) KONUMU KAYDET, 5) BAŞLANGICA GİT, 6) KAYITLI KONUMA GİT.

5.3. Görev 3 – HSV ile Renk Algılama

OpenCV kullanılarak, kameradan alınan canlı görüntüde HSV renk uzayında kırmızı / yeşil / mavi renklerini tespit eden bir PYTHON programı yazılacaktır. Görüntü HSV'ye çevrilip her renk için maske çıkarılır; tespit edilen renk hem görüntü penceresinde (etiket) hem terminalde (RED / GREEN / BLUE) gösterilir. Üç renk de ayrı ayrı denenerek doğru tespit gösterilmelidir. İpucu: Kırmızı renk HSV'de özel bir durumdur; dikkatli inceleyiniz.

İstenen kanıtlar: Python kodu (renk_algila.py) ve her renk için birer ekran görüntüsü (kirmizi.png, yesil.png, mavi.png) — ekran görüntüsünde kamera penceresi, tespit etiketi ve TERMİNAL ÇIKTISI birlikte görünecek şekilde.

5.4. Görev 4 – MQTT ile Haberleşme

Aynı ağdaki iki bilgisayar arasında MQTT ile mesajlaşma kurulacaktır; bir bilgisayar robot kolu (yayıncı), diğeri aracı (dinleyici) temsil eder. 1. bilgisayara mosquitto broker kurulur ve Python (paho-mqtt) ile arac/yuk topic'ine RED / GREEN / BLUE mesajlarından biri gönderilir. 2. bilgisayar, broker'ın IP adresine bağlanıp aynı topic'e abone olur ve gelen mesajı terminale yazar. Mesajın diğer cihazda gerçekten alındığı gösterilmelidir.

İstenen kanıtlar: İki cihazın terminal ekran görüntüleri (yayinci.png, dinleyici.png; gönderim ve alım görünecek), her iki cihazda ipconfig çıktısı (aynı ağ kanıtı) ve **yayıncı/dinleyici kodları**.

5.5. Görev 5 – YOLO ile Tabela Tespiti Modeli

Yarışmacı; yaya geçidi, tümsek, hemzemin geçit ve park tabelalarını tanıyan bir YOLO nesne tespit modelini KENDİSİ EĞİTİR ve eğittiği modelle kameradan canlı tespit yapan bir Python programı yazar. Model; organizasyonca iletilen tabela görsellerinden yararlanılarak eğitilir (görseller tabelalar.zip olarak verilmiştir). Sınıf adları: yaya, tumsek, hemzemin, park. Modelin nerede ve hangi ortamda eğitildiği değerlendirmede dikkate alınmaz. NOT: Verilen görsellerdeki yaya geçidi, tümsek, hemzemin geçit ve park tabelaları eğitilecektir; görsellerde yer alan diğer tabela ve nesneler, modelin doğruluğunu artırmak amacıyla kullanılmıştır ve eğitilmesi beklenmez.

Standart eğitim ayarları (ZORUNLU): Tüm takımlar aynı model ve ayarlarla eğitim yapar; farklı sürüm veya boyutla (yolo26n/s/m/x, YOLOv8 vb.) eğitilen modeller değerlendirmeye alınmaz.

AYAR	DEĞER
Taban model	**yolo26l**
Epochs	**100**

AYAR	DEĞER
Batch Size	**-1 (otomatik)**
Image Size	**640**

İstenen kanıtlar: best.pt (eğitilmiş model dosyası; ismi DEĞİŞTİRİLMEZ), egitim_grafik.png (eğitim sonuç grafiklerinin ekran görüntüsü), her tabela için birer tespit ekran görüntüsü (tabela_yaya.png, tabela_tumsek.png, tabela_hemzemin.png, tabela_park.png; kutu + sınıf adı + güven skoru görünecek şekilde) ve tespit.py (canlı tespit kodu).

.engine dosyası İSTENMEMEKTEDİR; yalnızca .pt dosyası teslim edilecektir. Model, organizasyonun çekeceği tabela görüntüleriyle test edilecektir.

5.6. Görev 6 – PLC ve HMI Programı

Verilen bağlantı şeması temel alınarak;

3.1 de verilen senaryoya uygun olarak gerekli PLC ve HMI programları Tia Portal v14 veya üzeri bir sürümde tek bir proje olarak hazırlanacak ve Tia Portal arşiv dosyası .zap14 veya üstü olarak teslim edilecektir.

- Pano bağlantı şemasında sensör çalışma modu ve sürücü parametre ayarları verilmiştir.
- PLC input /output etiketlemeleri (etiket listesi), HMI etiket listesi yarışmacı tarafından senaryoya ve bağlantı şemasına uygun açık ve anlaşılır şekilde düzenlenmelidir.
- Programlama dili: Ladder (LAD) olmalıdır. SCL/FBD opsiyonel olarak fonksiyon blokları içinde kullanılabilir, ancak ana ağ Ladder olmalıdır.
- Her networkte uygun yorumlar eklenerek program anlaşılır hale getirilmelidir.
- Sistemin kontrolü hem butonlar hem de HMI üzerinden sağlanmalıdır.
- HMI tek ekran olarak düzenlenecektir. Ergonomi dikkate alınacaktır.
- Programlama yapılırken uluslararası standartlar esas alınmalıdır. (Renk paleti IEC 62682 uygun olmalıdır)

5.7. Görevlerde Kullanılacak Kütüphaneler ve Sürümler

Değerlendirme ortamıyla uyum için aşağıdaki sürümler birebir kullanılmalıdır. Özellikle paho-mqtt MUTLAKA 2.x olmalıdır (1.x API'si ile çalışmaz).

KÜTÜPHANE / ARAÇ / PROGRAM	SÜRÜM
Tia Portal	**V14 ve üzeri**
Python	**3.10**
OpenCV (cv2)	**4.5.4**
NumPy	**1.26.4**
Pillow (PIL)	**9.0.1**

KÜTÜPHANE / ARAÇ / PROGRAM	SÜRÜM
paho-mqtt	**2.1.0 (2.x zorunlu)**
pyserial	**3.5**
tkinter	**8.6 (Python ile gelir)**
mosquitto (broker)	**2.0.11**
ultralytics (YOLO)	**8.4.x**

Not: mosquitto varsayılan kurulumda yalnızca localhost'u dinler; başka cihazdan bağlantı için yapılandırmaya "listener 1883" ve "allow_anonymous true" satırları eklenmelidir. Arduino tarafında AccelStepper ve Servo kütüphaneleri kullanılabilir; kart Arduino Uno, seri hız 115200 baud'dur.

5.8. Teslim Formatı

Altı görevin tüm çıktıları, aşağıdaki klasör yapısına uygun TEK bir ZIP dosyası olarak KYS üzerinden teslim edilir. Dosya adı takım adınız olacaktır; küçük harflerle, boşluk ve Türkçe karakter kullanmadan yazınız (örnek: yildirim_takimi.zip).

```
takim_adi.zip
├── gorev1.zip  (Sistem Algoritması: algoritma.pdf — akış şeması + sözde kod)
├── gorev2.zip  (Arayüz + Step Motor: arayuz.py + motor.ino + konumlar.json)
├── gorev3.zip  (HSV Renk Algılama: renk_algila.py + 3 ekran görüntüsü)
├── gorev4.zip  (MQTT: yayinci.py + dinleyici.py + 2 ekran görüntüsü)
├── gorev5.zip  (YOLO: best.pt + egitim_grafik.png + tespit.py + 4 tespit görüntüsü)
├── gorev6.zip  (PLC ve HMI: TIA Portal proje arşivi)
└── README.txt (nasıl çalıştırılır, kısa açıklama)
```

Her klasörün içeriği, ilgili görev bölümündeki "İstenen kanıtlar" listesine uygun olmalıdır. Video teslimi istenmez. Teslim edilen kodların, Bölüm 5'te tanımlı standart bağlantı ve ayarlarla çalışır durumda olması zorunludur. Tüm teslimlerde özgünlük (intihal, kopya, yetkisiz kullanım) kontrolü yapılacaktır.

6. PUANLAMA

Yarı final puanlaması tamamen görevler üzerinden yapılır. Her görev; istenen çıktıların eksiksizliği, çalışırlık (Görev 2 kodu gerçek step motor düzeneğinde, Görev 5 modeli gerçek tabela görüntüleriyle test edilir) ve tasarım kalitesi (karar sorularının karşılanması) ölçütlerine göre değerlendirilir.

GÖREV	PUAN
Görev 1 – Sistem Algoritması Tasarımı (Kol + Araç)	**20**
Görev 2 – Arayüz ve Step Motor Kontrolü	**10**
Görev 3 – HSV ile Renk Algılama	**10**
Görev 4 – MQTT ile Haberleşme	**10**
Görev 5 – YOLO Tabela Tespiti Modeli	**20**
Görev 6 – PLC ve HMI Programı	**30**
TOPLAM	**100**

Final aşamasının puanlama esasları, final parkur düzeniyle birlikte organizasyon tarafından ayrıca duyurulacaktır.

7. FİNAL AŞAMASI HAKKINDA

Final sahası; konveyör, robot kol, otonom araç, PLC ve HMI dahil tüm sistemler organizasyon tarafından KURULU ve ÇALIŞIR durumda hazır bulundurulur. Finale kalan takımlara; PLC, robot kol ve otonom araç yazılım paketleri ile detaylı final senaryosu, final öncesinde KYS üzerinden iletilir ve aynı kodlar final günü donanımlarda kurulu gelir. Final günü takımlardan; çalışır sistem üzerinde REVİZYONLAR yapmaları ve otonom araç koduna, o gün duyurulacak EK GÖREVİ eklemeleri istenir. Finale ilişkin tüm detaylar (saha düzeni, senaryo akışı, süre ve puanlama esasları) finale kalan takımlara ayrıca duyurulacaktır.

8. KURALLAR

- Teslimler KYS üzerinden, Bölüm 5.8'de tanımlı yapıya uygun TEK bir ZIP dosyası olarak yapılır. Formata uymayan veya eksik teslimler değerlendirmeye alınmayabilir.
- Tüm teslimlerde özgünlük (intihal, kopya, yetkisiz kullanım) kontrolü yapılır; ihlal tespitinde takım diskalifiye edilir.
- Görev 2'deki standart bağlantıya ve Görev 5'teki standart eğitim ayarlarına uymayan çalışmalar değerlendirilmez.
- Bu şartname ile Genel Şartname arasında çelişki olması hâlinde Genel Şartname esas alınır.

9. İLETİŞİM

Bu teknik şartnameye ilişkin sorular, TEKNOFEST Yarışmalar Komitesi tarafından duyurulan resmi iletişim kanalları (yarışma e-posta grubu ve iletisim@teknofest.org adresi) üzerinden iletilmelidir.

Sorular kategori bazlı olarak değerlendirilir.